

McGill



MBZUAI

NUTANIX

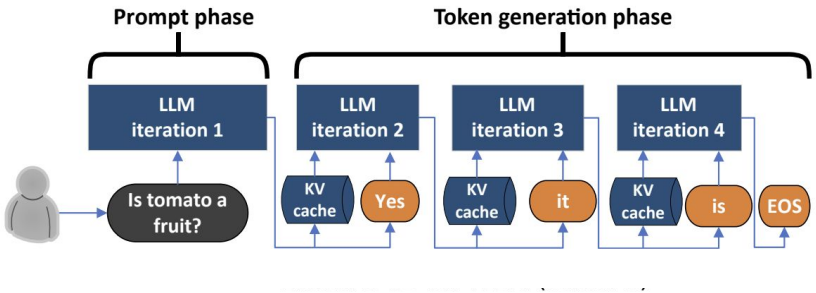
DeltaServe: Host-Agnostic Co-Serving of Inference and Fine-Tuning for LLMs

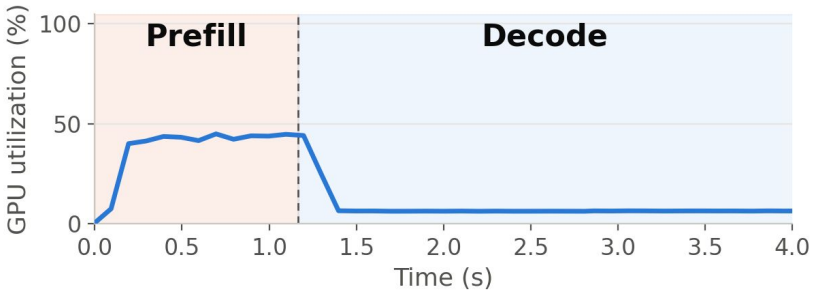
Jiaxuan Chen[1], Jianshu She[2], Rajat Ghosh[3],
Karan Gupta[3], Qirong Ho[2], Xue Liu[1], Oana Balmau[1]

1. McGill University *2. MBZUAI* *3. Nutanix Inc.*

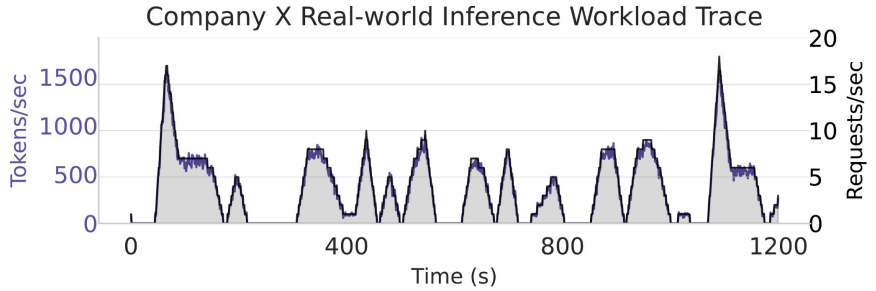
GPU Underutilization in LLM Serving

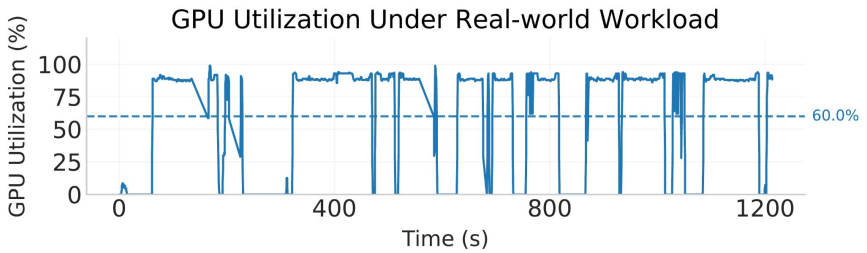
Problem 1: Autoregressive Execution





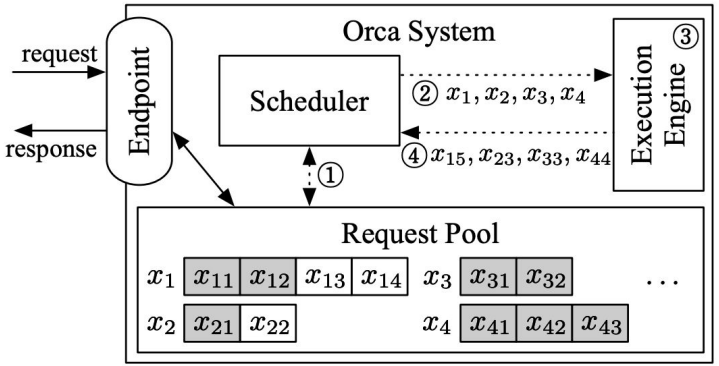
Problem 2: User-Driven Traffic



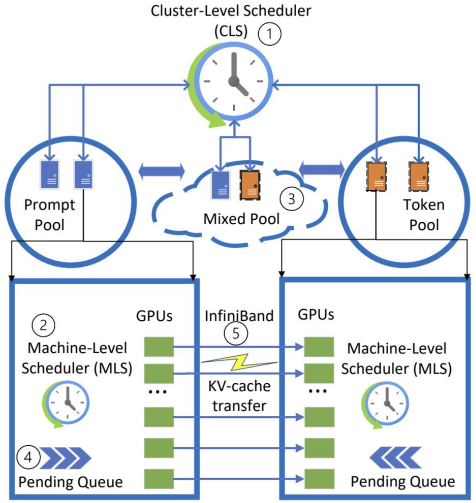


GPU capacity is underutilized both during active inference and between traffic bursts.

Prior Direction: Improving the Inference Pipeline



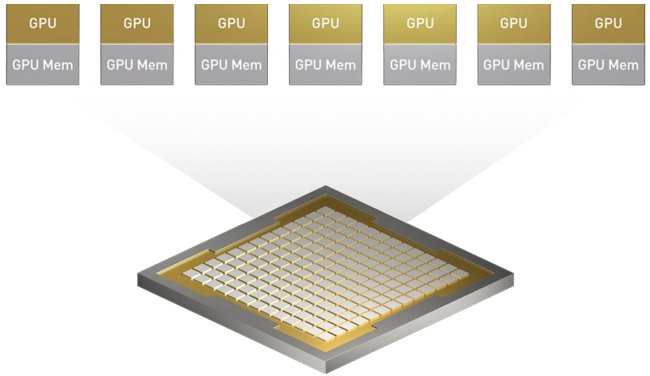
Orca (OSDI 2022) proposed prefill/decode mixed batching



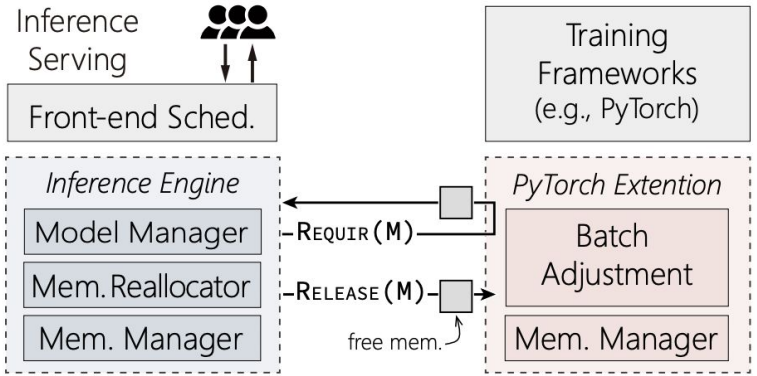
Splitwise (ISCA 2024) proposed phase disaggregation

Limitation: These systems improve efficiency while requests are available. When user traffic falls, the inference GPU pool is still underutilized.

Prior Direction: GPU Sharing



NVIDIA GPU-sharing mechanisms, such as MIG and MPS, enable multiple workloads to share a GPU through resource partitioning or concurrent execution.



SIRIUS (USENIX ATC ’25) Co-locates inference engine and training frameworks through spatial sharing and fast GPU-memory handover.

Limitation: Inference demand changes at sub-second timescales, faster than generic GPU sharing can safely adapt. Sharing peak-provisioned capacity risks inference SLO violations during bursts.

Background: Low-Rank Adaptation (LoRA)

Reduce the number of parameters required to fine-tune by a factor of 100 - 10,000.

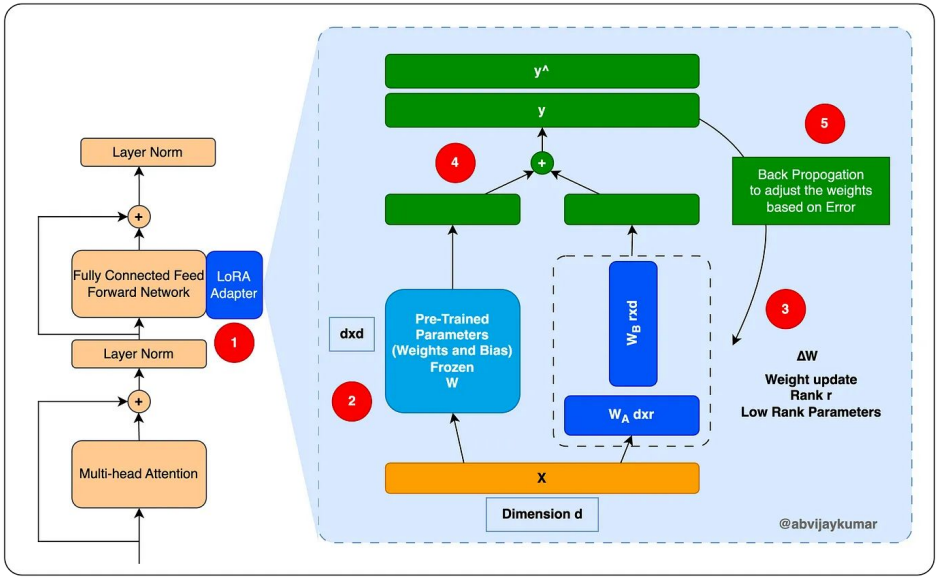
Low Rank Adaptation: freeze the pre-trained **base model** and update a small number of additional parameters.

$$h = xW' = x(W + AB)$$
$$= xW + xAB.$$

Where $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and $r \ll \min(d_{\text{in}}, d_{\text{out}})$

$A \in \mathbb{R}^{d_{\text{in}} \times r}$

$B \in \mathbb{R}^{r \times d_{\text{out}}}$

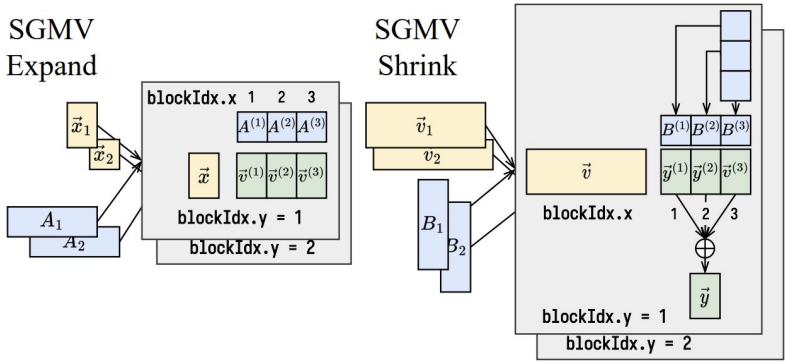


Background: Multi-LoRA Serving Systems

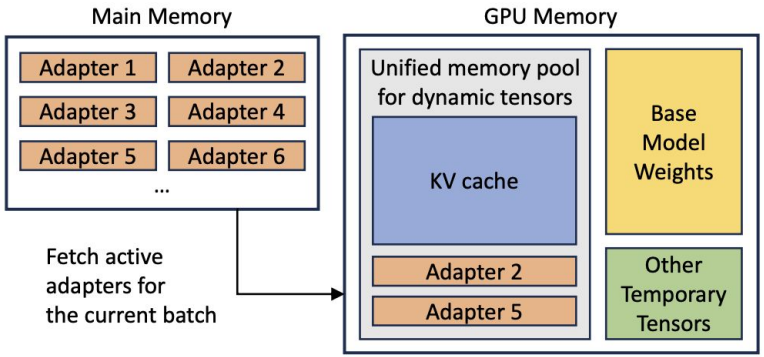
LoRA also changes how specialized models are served:

- **Initially**, LoRA adapters were merged into the base model to eliminate additional computation during inference.
- **As demand grew** for task-specific models, maintaining a separate model for every adapter became inefficient.
- **Multi-LoRA serving systems**, such as Punica and S-LoRA, batch inference requests from multiple adapters over a shared base model.

Punica's CUDA kernel for multi-LoRA batching



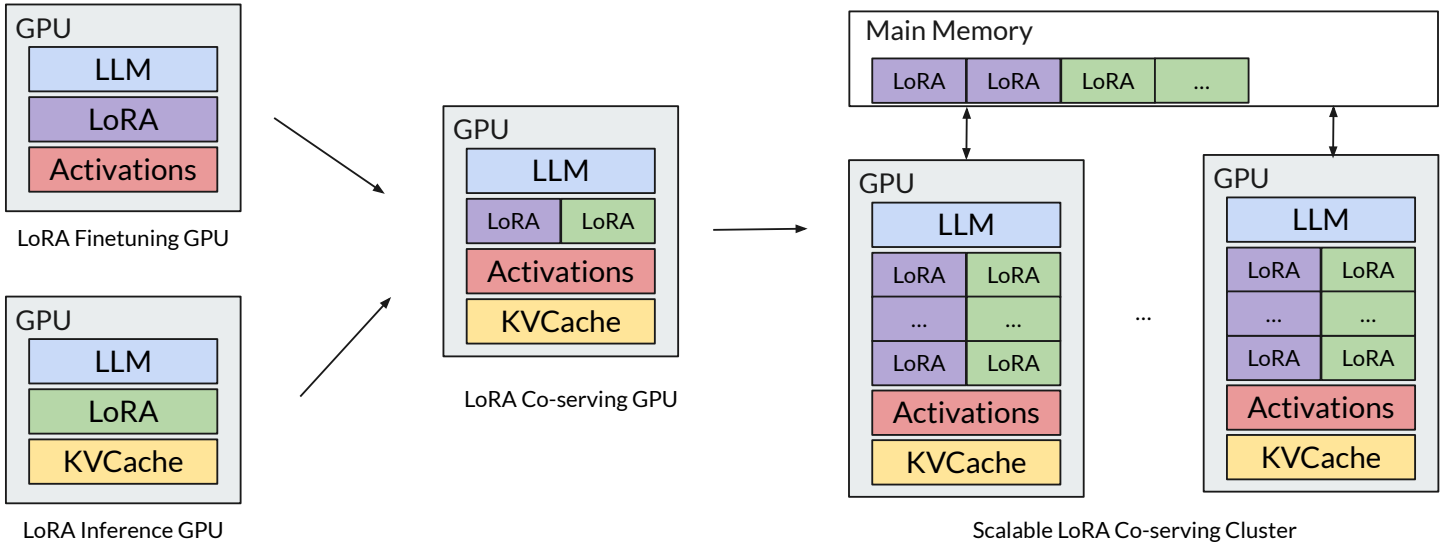
S-LoRA's unified memory for adapters weight and KV Cache



Background: Co-Serving Inference and Fine-Tuning

Building on LoRA, inference and fine-tuning can efficiently share the same GPU:

- Both workloads reuse the same **frozen base model**.
- The fine-tuning forward pass is structurally identical to inference prefill, except that **activations** are retained for backward. Only lightweight LoRA adapters needs update.



Challenges

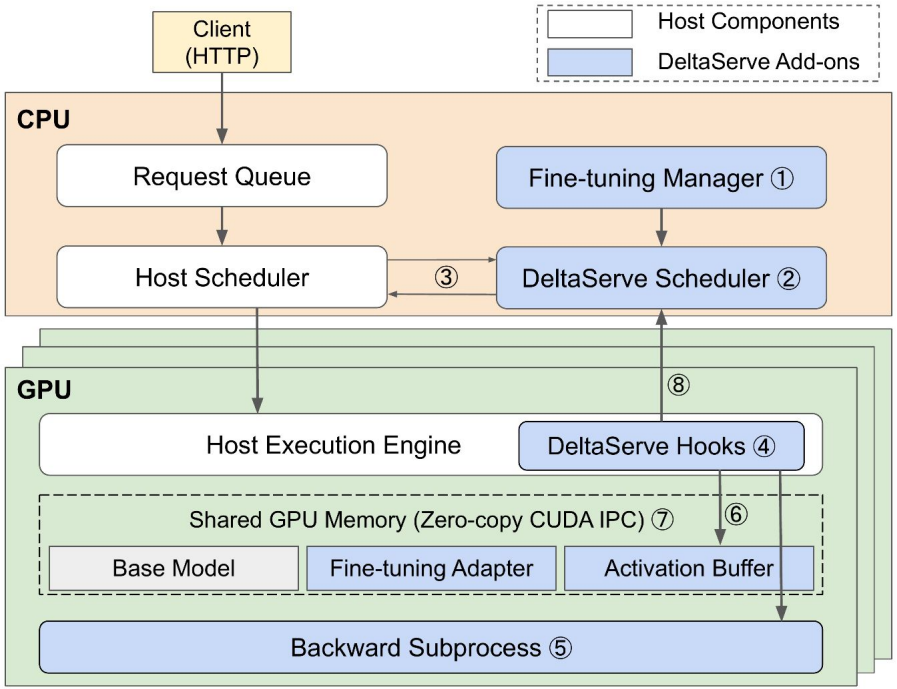
Principle: latency-critical, high priority **Inference**; best-effort, transparent **fine-tuning.**

- **SLO interference:** Fine-tuning competes with latency-critical inference for GPU resources.
- **Backward execution:** Long-running training must yield promptly to inference.
- **Dynamic demand:** Fine-tuning admission must adapt to sub-second changes in inference load.
- **Practical integration:** Production relies on mature inference engines such as vLLM and SGLang. A practical co-serving solution should extend these engines rather than replace them.

We present **DeltaServe**: **host-agnostic** co-serving framework that adds best-effort LoRA fine-tuning to existing inference engines while preserving inference SLOs.

DeltaServe Overview

DeltaServe is design to be host-agnostic extension to existing inference engines



Modern inference engines differ internally, but share a common serving pipeline.

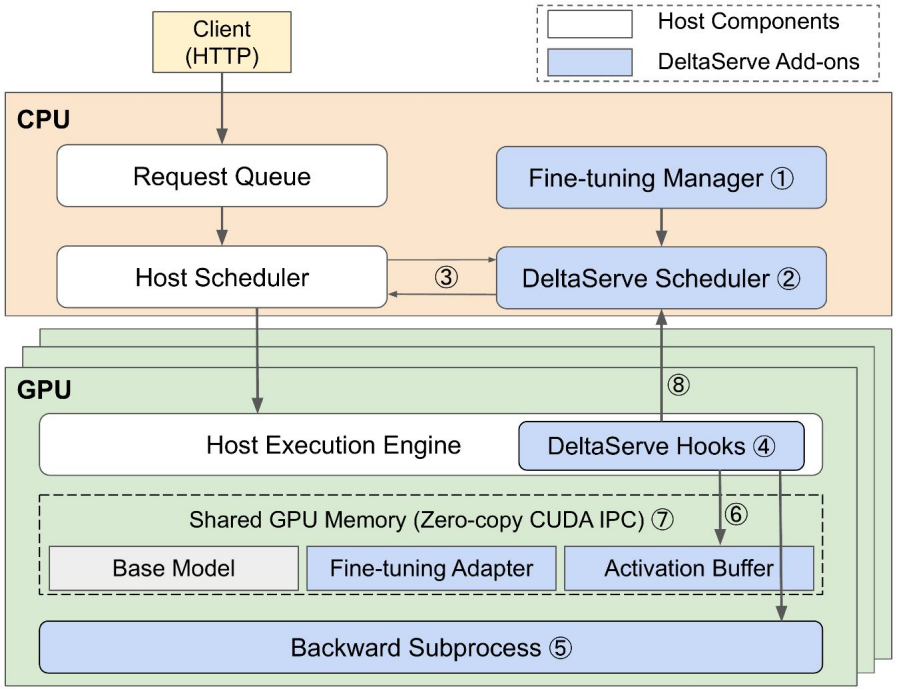
- **White boxes**: generic host components.
- **Blue boxes**: DeltaServe add-ons.

This separation allows:

- DeltaServe to remain **host-agnostic**; only integration hooks are host-specific.
- DeltaServe to require only one host capability: **multi-LoRA batching**.
- The host to retain its optimized inference execution.

DeltaServe Overview

CPU Control Plane: Add best-effort fine-tuning work only when inference has sufficient SLO headroom.



① Fine-Tuning Manager — Prepare

- Loads and tokenizes the training data.
- Tracks training progress.

② DeltaServe Scheduler — Decide

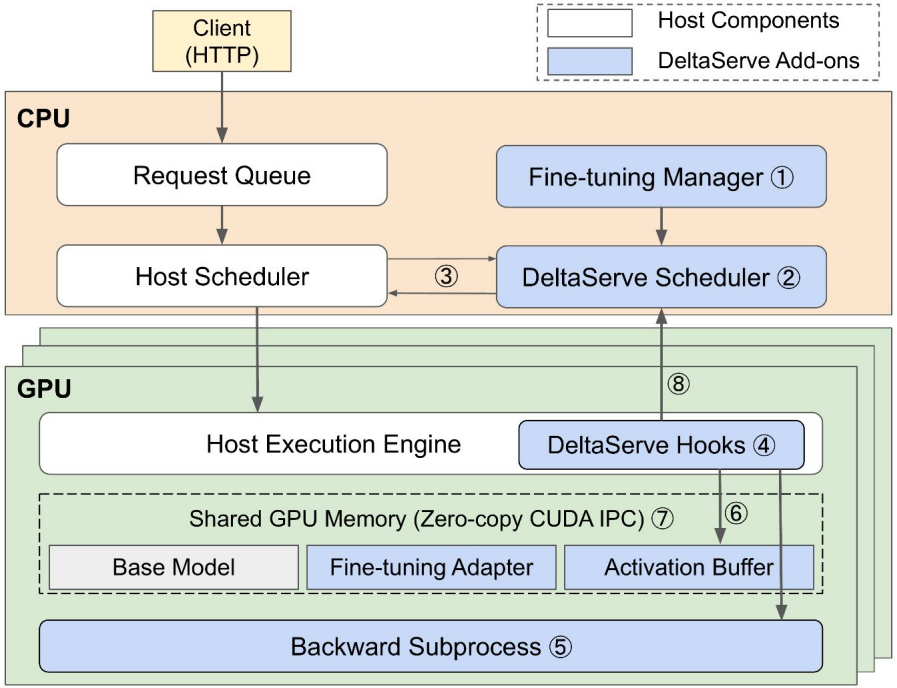
- intercept the host's proposed inference batch before dispatched to GPU.
- Admits fine-tuning samples that fit within the inference SLO budget.

③ Host Scheduler Interface — Handoff

- Creates one mixed inference and fine-tuning batch.
- Fine-tuning samples retire after one forward pass and produce no client-visible output.

DeltaServe Overview

GPU Execution Plane: Reuse the host forward path while isolating backward execution**.**



④ **DeltaServe Hooks**: Lightweight hooks capture the activations during the host forward pass.

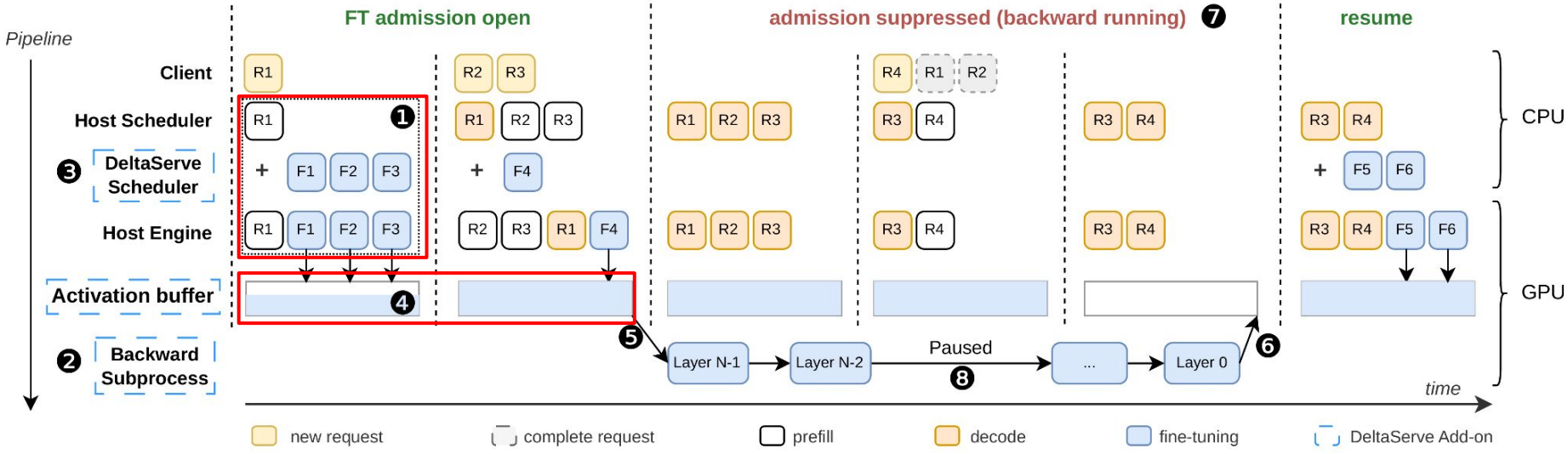
⑤ **Backward Subprocess**: compute gradients and update the adapter outside the host inference engine.

⑥ + ⑦ **Memory Management:** Zero-copy CUDA IPC that shares the base model, adapter, and activation buffer between the inference and backward subprocess.

DeltaServe's Co-serving Model

Mixed forward batch ❶:
Each admitted fine-tuning sample becomes a **single-step, prefill-only** request. It shares the host's multi-LoRA forward batch with inference.

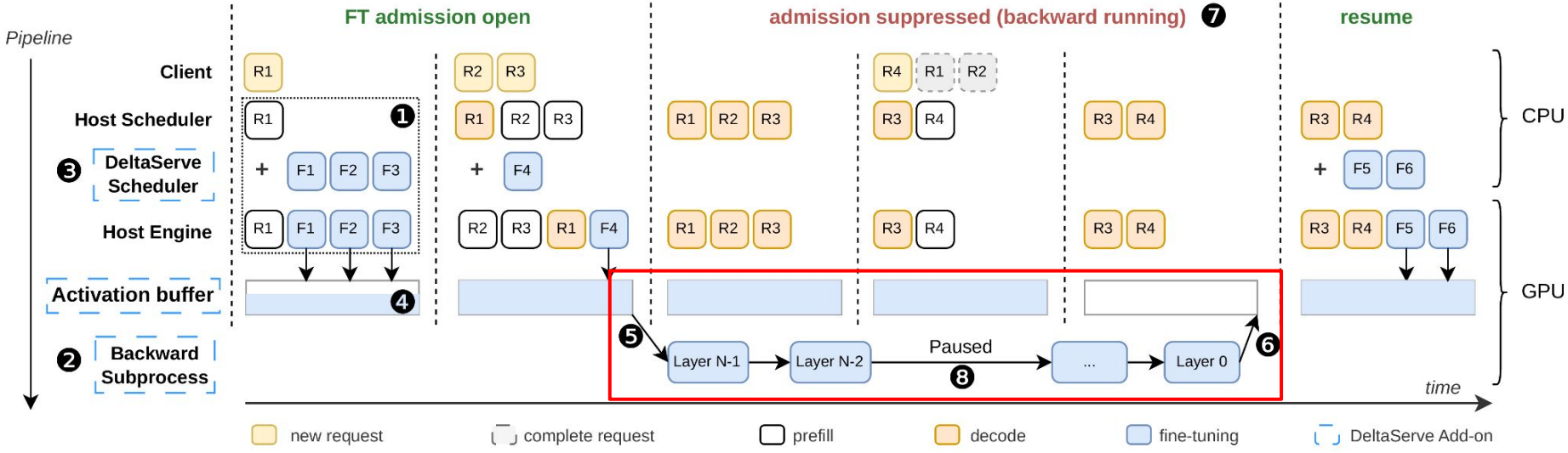
Activation Accumulation ❹:
Capture activations only for the fine-tuning samples. Activations accumulate across forward steps until enough tokens are available for one backward pass.



DeltaServe’s Co-serving Model

Backward Subprocess:

- **Start ❺:** Once the activation buffer is full, backward begins and fine-tuning admission is closed ❼.
- **Yield ❽:** Backward pauses at a layer boundary whenever the inference batch contains prefill. It resumes after prefill finishes.
- **Complete ❻:** After updating the adapter, DeltaServe releases the consumed activations and reopens fine-tuning admission.



DeltaServe Scheduler ❸

Question: How much fine-tuning workload can be admitted to an inference step without violating inference SLO?

Key Idea: before issuing an inference batch to the GPU, DeltaServe computes the latency headroom remaining after serving inference: $H_{\mathrm{FT}} = \min(\Delta_{\mathrm{TTFT}}, \Delta_{\mathrm{TPOT}}) - \widehat{T}(B_{\mathrm{inf}})$

where **Δ_{TTFT}**: Time remaining before any request in the batch would violate its TTFT target.

Δ_{TPOT}: Time remaining before any active decode request would violate its TPOT target.

$\widehat{T}(B_{\mathrm{inf}})$: Predicted execution time of the proposed inference-only batch.

H_{FT}: Remaining time available for additional fine-tuning work.

Therefore, using this headroom safely and efficiently requires an accurate predictor of mixed-batch execution time.

Analytic Execution-Time Predictor

DeltaServe uses a **analytical execution-time predictor** derived from the computational structure of Transformer models.

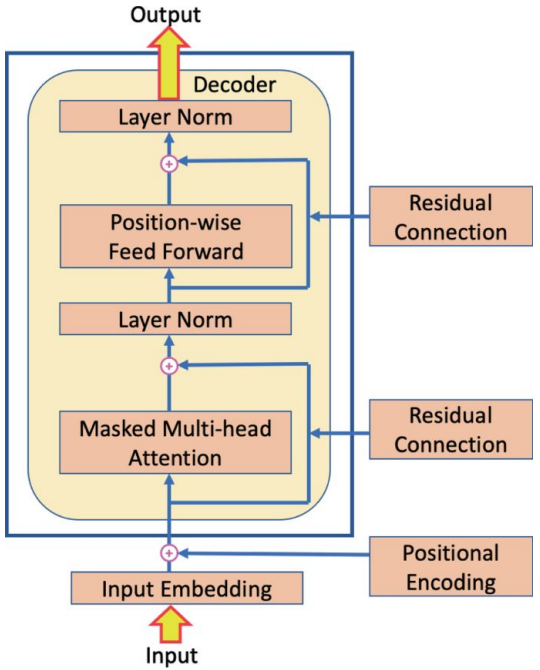
For a Transformer model with hidden dimension d, a single sequence L's forward cost, is dominated by:

- **Attention**: cost grows quadratically with the sequence lengths processed in the step.

$$C_{\text{attention}}(L) = O(L^2 d) \propto L^2$$

- **FFN layers**: cost grows linearly with the total number of processed tokens.

$$C_{\text{FFN}}(L) = O(L\, d\, d_{\text{ff}}) \propto L$$



Analytic Execution-Time Predictor

At serving time, the model architecture is **fixed**. Therefore, the execution time of a forward step is determined primarily by its **batch composition**.

For an inference–fine-tuning mixed batch, the batch contains:

- n_i: prompt length of prefill request i
- $\Sigma_i n_i$: total number of prefill tokens
- B_d: number of active decode requests
- T_{ft}: number of admitted fine-tuning tokens
- K: total KV-cache size of the batch

Therefore, DeltaServe estimates the batch execution time as:

$$T \approx \alpha \sum_i n_i^2 + \beta \left(\sum_i n_i + B_d \right) + \gamma T_{\text{ft}} + \varepsilon K + c$$

Scheduling Algorithm

DeltaServe uses the predictor to add fine-tuning samples without violating the inference SLO.

Lines 2–6: Check availability: check if fine-tuning admission is open.

Lines 8–13: Protect inference: compute the TTFT/TPOT budget.

Lines 14–21: Add samples greedily: shorter samples first and stop when either the latency budget or buffer capacity is exhausted.

Lines 23–24 — Commit: Reserve activation-buffer space and enqueue the selected samples into the host batch.

Algorithm 1 Fine-tuning admission for one step.

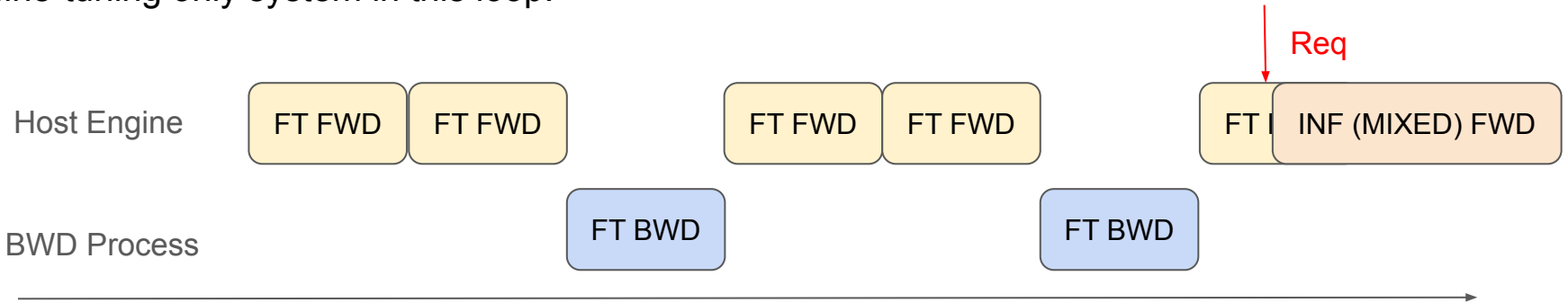
Inputs: Host proposed inference batch B_{inf} for the next step; SLO estimator E; fine-tuning pool Q_{ft}; Backward process state P_B; remaining buffer $S_{\max}$; host capability *mixedBatch*

Outputs: Fine-tuning samples R_{ft} to enqueue for the next step

1: $R_{\text{ft}} \leftarrow [\,]$
2: **if** P_B is running **or** $S_{\max} \le 0$ **or** $Q_{\text{ft}} = \emptyset$ **then**
3: **return** R_{ft}
4: **end if**
5: **if not** *mixedBatch* **and** B_{inf} has no prefill tokens **then**
6: **return** R_{ft} cannot merge fine-tuning into a decode-only step
7: **end if**
8: $\Delta_{\text{tt}} \leftarrow$ TTFT budget of B_{inf} Eq. *III*, with queue wait
9: $\Delta_{\text{tp}} \leftarrow T_{\text{tpot}} - E.\textsc{predict}(D_{\text{wait}})$ D_{wait} empty if decode shares the step
10: $\Delta \leftarrow \min(\Delta_{\text{tt}}, \Delta_{\text{tp}})$
11: **if** $E.\textsc{predict}(B_{\text{inf}}) > \Delta$ **then**
12: **return** R_{ft} no slack even without fine-tuning
13: **end if**
14: **for** $ft \in Q_{\text{ft}}$ in increasing token length **do**
15: $B' \leftarrow B_{\text{inf}} \cup R_{\text{ft}} \cup \{ft\}$
16: $c_1 \leftarrow \textsc{tokens}(R_{\text{ft}}) + \textsc{tokens}(ft) > S_{\max}$ activation buffer full
17: $c_2 \leftarrow E.\textsc{predict}(B') > \Delta$ would violate an SLO
18: **if** $c_1 \vee c_2$ **then**
19: **break**
20: **end if**
21: $R_{\text{ft}}.\textsc{append}(ft)$
22: **end for**
23: $\textsc{Claim}(R_{\text{ft}})$ reserve buffer, remove from pool
24: **return** R_{ft}

Fine-tuning Preemption

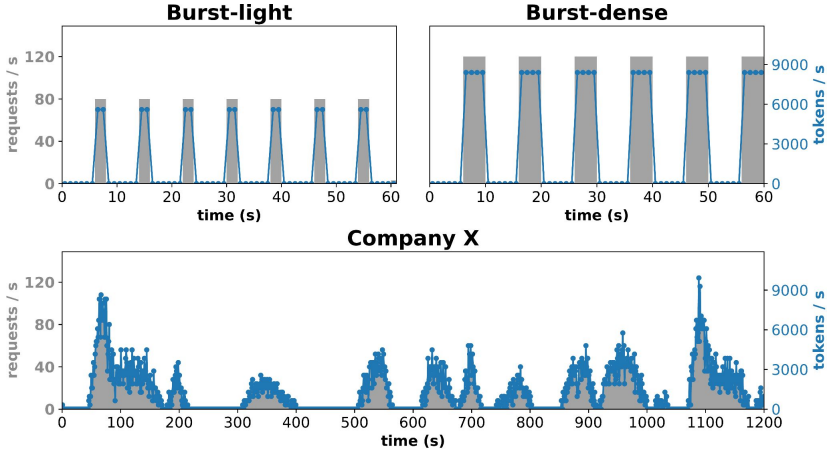
Therefore, in a DeltaServe-enabled system, at inference idle period, the system will run as a fine-tuning only system in this loop:



However, what if inference arrives during a fine-tuning forward, when the host engine is occupied?

DeltaServe abort the fine-tuning only batch, discards the activations and immediately schedule inference. The interrupted fine-tuning samples return to the pool and are admitted again later.

Evaluation: Experiment Setup



Environment

- **Model**: Llama 3-8B,
- **Hardware**:
  - 4× A100 40 GB, 24-core Xeon, 340 GB DRAM.
  - RTX 5090 32 GB, Ryzen 9 9950X, 64 GB DRAM.
- **Host engines**: vLLM, SGLang, and S-LoRA.

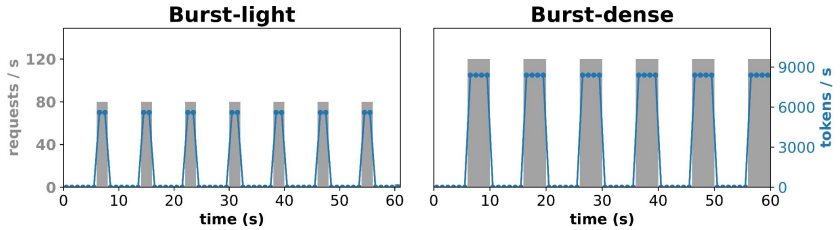
Workloads

- **Inference**: Burst-light, Burst-dense, and.
  - Burst-light: 2-second peaks at 80 RPS.
  - Burst-dense: 4-second peaks at 120 RPS.
  - 20-minute Nutanix production trace
- **Fine-tuning**: Alpaca dataset with LoRA rank 16, α = 32, and dropout 0.05 on attention

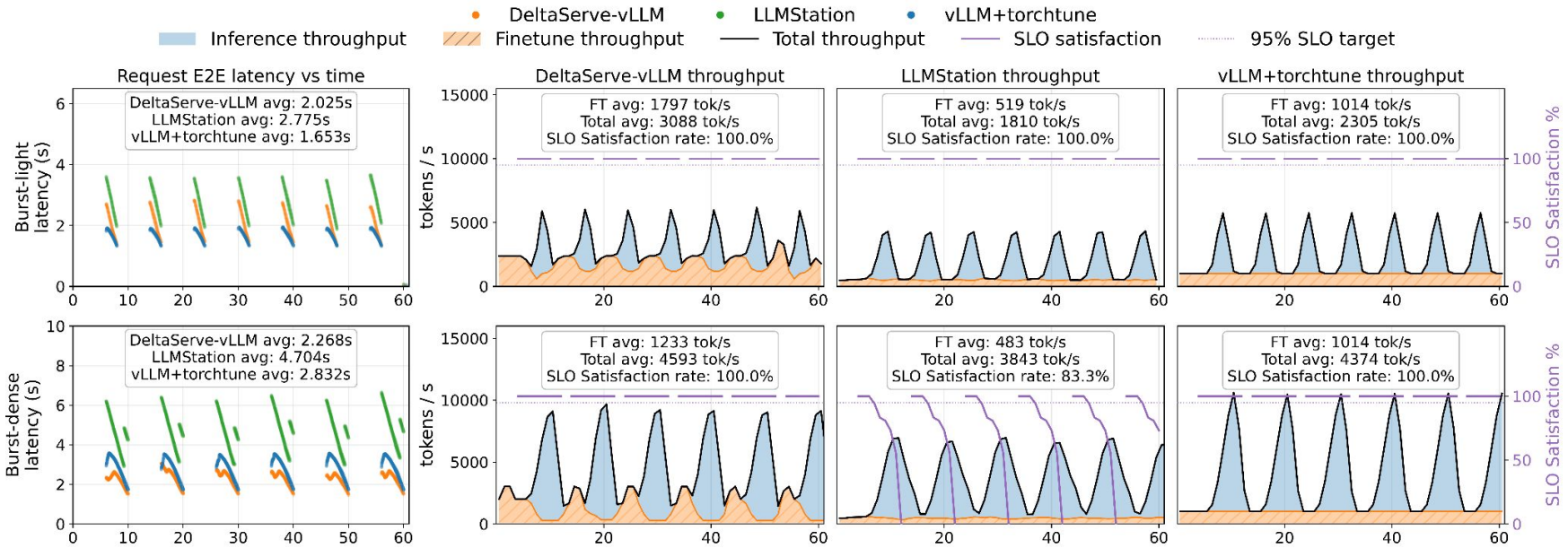
Baselines:

- LLMStation: state-of-the-art decode-side co-serving.
- vLLM + torchtune: inference and fine-tuning on separate GPU pools.

Evaluation: End-to-end Comparison



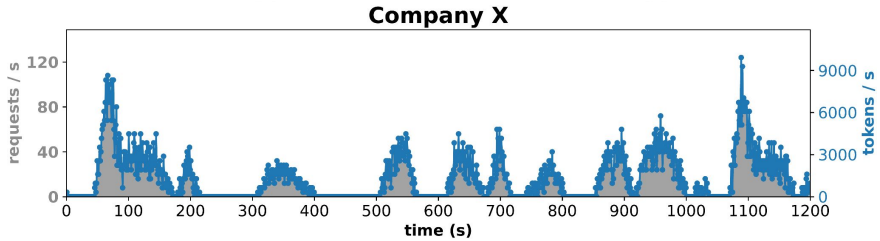
DeltaServe continuously **adapts** fine-tuning throughput to changing inference headroom.
This dynamic sharing preserves inference SLOs while harvesting more GPU capacity for fine-tuning than either baseline.

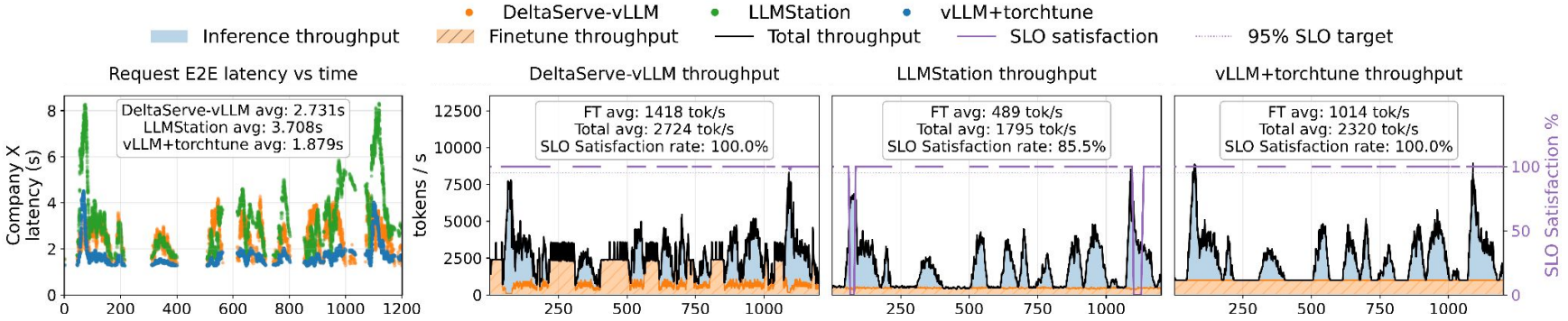


Evaluation: End-to-end Comparison

Production Workload

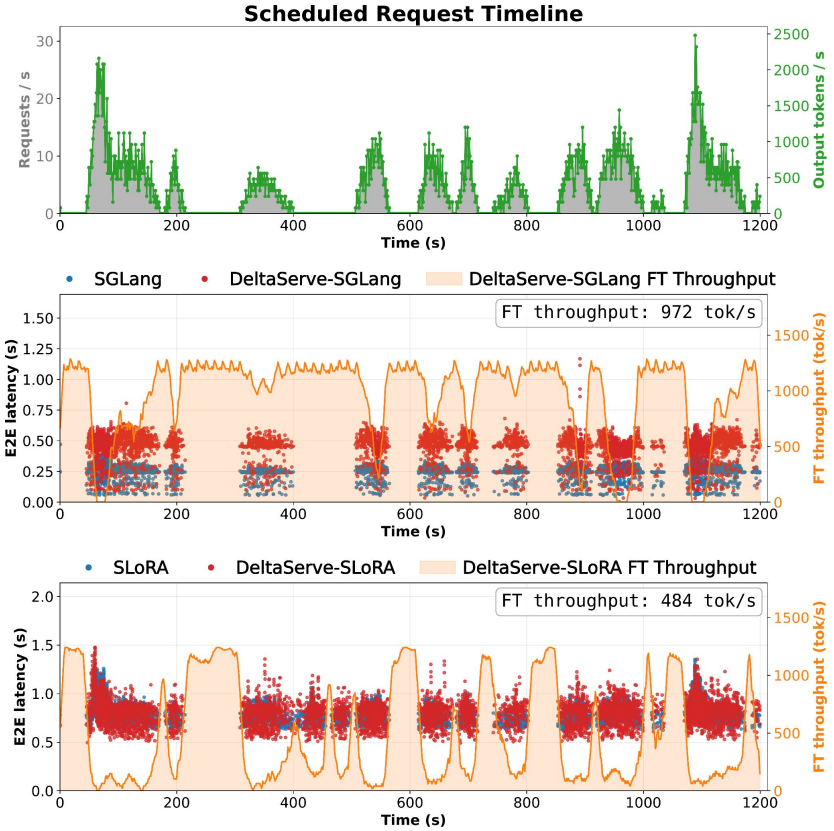
- Irregular, heavy-tailed traffic with bursts.
- Extended low-load intervals are interrupted by sudden spikes above 100 RPS.
- The available inference headroom changes rapidly and unpredictably.





Across both synthetic bursts and the irregular production trace, DeltaServe increases fine-tuning during idle periods and suppresses it as inference demand rises.

DeltaServe-SGLang & DeltaServe-S-LoRA



Validate portability across different host-engines:

- **SGLang**: a production-oriented engine with an optimized scheduling and execution pipeline.
- **S-LoRA**: a lightweight engine close to DeltaServe's minimum requirement: multi-LoRA batching.

Insights:

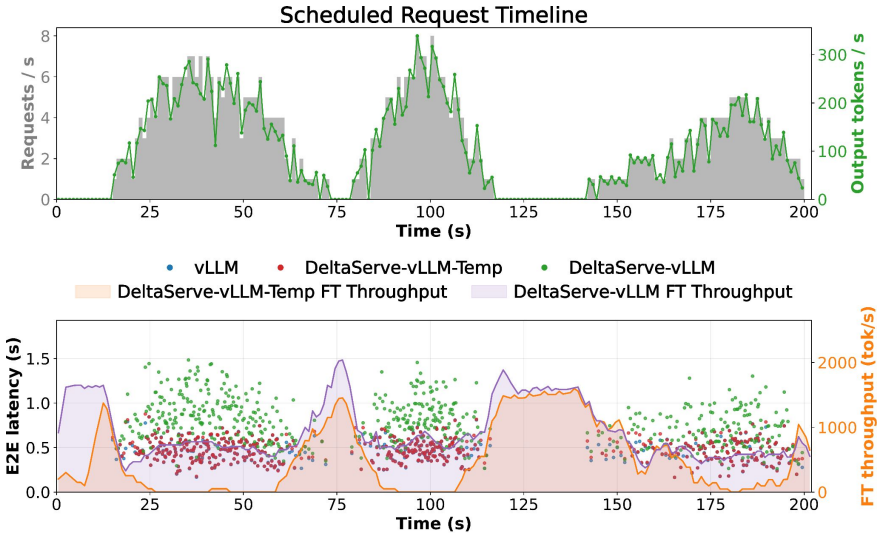
- The same DeltaServe core preserves inference SLOs and harvests fine-tuning capacity on both hosts.
- Different fine-tuning rates reflect the amount of headroom left by each host's inference efficiency.

Ablation Study: Forward Batch Co-serving

Question: How much does forward-batch co-serving improve fine-tuning throughput?

Configurations:

- **vLLM**: inference only.
- **DeltaServe-vLLM**: full system.
- **DeltaServe-Temp**: limit fine-tuning to run only when the host has no inference work.



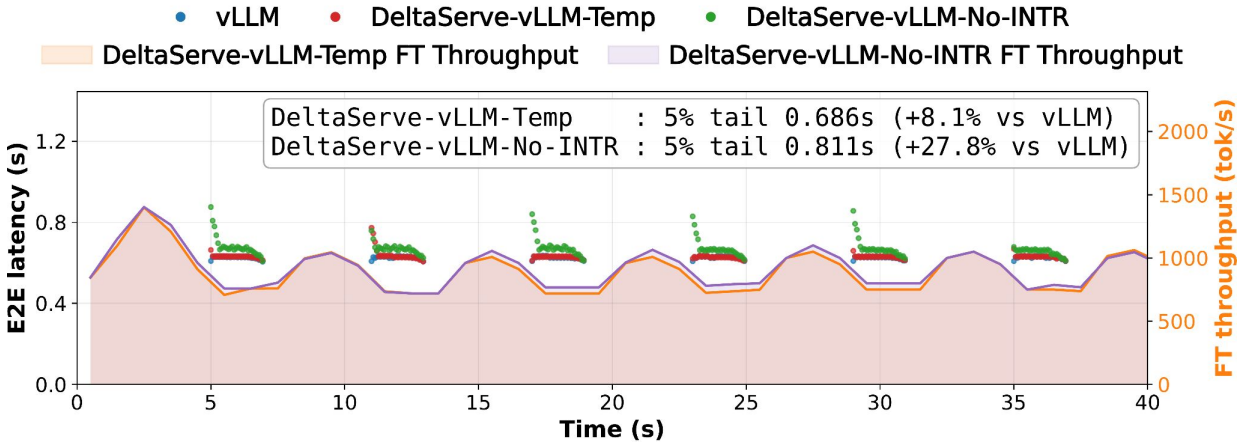
Insight: The gap between DeltaServe-Temp and full DeltaServe is a tunable operating space: the SLO budget controls how much inference headroom is allocated to fine-tuning, trading inference latency for fine-tuning throughput.

Ablation Study: Fine-tuning forward interruption

Question: Does interrupting fine-tuning-only forward passes protect inference latency?
Workload: Scaled burst-light trace on one RTX 5090.

Configurations:

- **vLLM**: inference only.
- **DeltaServe-Temp**: limit fine-tuning to run only when the host has no inference work.
- **DeltaServe-No-INTR**: disable fine-tuning only batch interruptibility.



Insight: fine-tuning only batch interruption protects inference tail latency with almost no loss in fine-tuning throughput.

Next Steps:

1. **Paper Submission**
    - DeltaServe is now available on arXiv.
    - Preparing a submission to EuroSys or MLSys 2027.
2. **Scaling to Larger Models**
    - Add **tensor-parallelism fine-tuning** to DeltaServe.
    - Replace the current main result with an end-to-end evaluation of a larger LLM model.
3. **From Co-Serving to Self-Evolving Agents**
    - Jianshu She plans to build an **self-evolving system for agentic workloads** on DeltaServe.
    - Agents will learn continuously from tool-use trajectories, environment feedback, and verified outcomes generated during serving.
    - The system will convert useful experience into LoRA updates, validate candidate adapters, and deploy only improvements that preserve inference SLOs.

Takeaways

1. **GPU headroom is unavoidable and dynamic.**

   Autoregressive execution and bursty user traffic leave changing amounts of unused capacity, since inference deployments are provisioned for peak demand.

2. **DeltaServe harvests this capacity without replacing the serving engine.**

   It adds SLO-aware admission, mixed forward batches, and preemptible fine-tuning to existing multi-LoRA engines such as vLLM, SGLang, and S-LoRA.

3. **Inference remains the priority**.

   Fine-tuning expands and contracts with the availab le SLO headroom, improving fine-tuning throughput while preserving inference SLOs.

DeltaServe turns otherwise-idle inference capacity into useful fine-tuning work.